

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 04 – ENKAPSULASI



NAMA: CALYA ALESITA PUTRI IRAWAN

NIM: 103132400020

KELAS: PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Enkapsulasi merupakan salah satu konsep dalam pemrograman berorientasi objek yang dapat diterapkan dengan menggunakan access modifier seperti *public*, *protected*, dan *private*. Modifier digunakan untuk mengatur akses terhadap class, method, dan variable. Dalam penerapannya, atribut biasanya dibuat *private*, sedangkan method dibuat *public* agar data tidak diakses secara langsung dan dapat dikelola melalui method seperti getter dan setter.

Constructor merupakan method khusus yang digunakan untuk membuat atau menginisialisasi sebuah objek. Nama constructor sama dengan nama class dan tidak memiliki nilai return. Java dapat memiliki beberapa constructor dengan parameter yang berbeda, yang disebut overloading constructor. Constructor juga dapat digunakan untuk langsung memberikan nilai awal pada atribut ketika objek dibuat.

Package digunakan untuk mengelompokkan class berdasarkan kategori tertentu sehingga struktur program lebih teratur dan dapat membantu menghindari konflik penamaan. Jika class yang digunakan berada pada package yang berbeda, maka diperlukan *import* agar class dapat digunakan.

B. Guided

1. Guided 1: Constructor

```
package project.guided1.Manusia;

public class Manusia {

    private String nama;
    private int umur;

    public Manusia() {
    }

    public Manusia(String nama) {
        this.nama = nama;
    }

    public Manusia(String nama, int umur) {
        this.nama = nama;
        this.umur = umur;
    }

    public void setNama(String nama) {
        this.nama = nama;
    }
}
```

```

    public String getNama() {
        return nama;
    }

    public void setUmur(int umur) {
        this.umur = umur;
    }

    public int getUmur() {
        return umur;
    }
}

```

```

package project.guided1.main;

import project.guided1.Manusia.Manusia;

public class DemoManusia {
    public static void main(String[] args) {

        Manusia arrMns[] = new Manusia[3];

        //Constreuctor pertama
        Manusia objMns1 = new Manusia("Markonah", 76);

        // kedua
        Manusia objMns2 = new Manusia("John");

        //ketiga
        Manusia objMns3 = new Manusia("Hoshi", 44);

        arrMns[0] = objMns1;
        arrMns[1] = objMns2;
        arrMns[2] = objMns3;

        for (int i = 0; i<3; i++) {
            System.out.println("Nama: " + arrMns[i].getNama());
            System.out.println("Umur: " + arrMns[i].getUmur());
            System.out.println();
        }
    }
}

```

Output :

```
Nama: Markonah
Umur: 76

Nama: John
Umur: 0

Nama: Hoshi
Umur: 44
```

2. Guided 2: Package

```
package project.guided2.HargaPulsa;

public class HargaPulsa {
    public void info(){
        System.out.println("Harga Pulsa");
        System.out.println("Rp. 5000");
        System.out.println("Rp. 10000");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
    }
}
```

```
package project.guided2.HargaToken;

public class HargaToken {
    public void info(){
        System.out.println("Harga Token");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
        System.out.println("Rp. 200000");
        System.out.println("Rp. 500000");
        System.out.println();
    }
}
```

```
package project.guided2.Main;

import project.guided2.HargaPulsa.HargaPulsa;
import project.guided2.HargaToken.HargaToken;
```

```

public class Main {
    public static void main(String[] args){
        HargaToken objectToken = new HargaToken();
        objectToken.info();
        HargaPulsa objectPulsa = new HargaPulsa();
        objectPulsa.info();
    }
}

```

Output :

```

Harga Token
Rp. 20000
Rp. 50000
Rp. 100000
Rp. 200000
Rp. 500000

Harga Pulsa
Rp. 5000
Rp. 10000
Rp. 20000
Rp. 50000
Rp. 100000

```

C. Unguided

1. Soal Unguided 1

Pada Unguided Modul 4, dibuat program pengelolaan dataset dengan menerapkan konsep enkapsulasi, constructor overloading, package, getter dan setter. Program terdiri dari class Dataset untuk menyimpan informasi dataset seperti nama, jumlah baris, jumlah kolom, dan jumlah data missing. Selanjutnya dibuat class LaporanDataset untuk menampilkan informasi dan status dataset berdasarkan persentase missing, serta class Main untuk membuat beberapa objek dataset menggunakan constructor yang berbeda, menyimpannya dalam array, dan menampilkan hasil laporan. Program ini juga menerapkan pemisahan class ke dalam beberapa package agar struktur program lebih terorganisir.

Jawaban :

```

package project.unguided.model;

public class Dataset {

    private String nama;

```

```
private int jumlahBaris;
private int jumlahKolom;
private int jumlahMissing;

public static final double BATAS_MISSING = 5.0;

private static int totalDataset = 0;

// Constructor tanpa parameter
public Dataset() {
    totalDataset++;
}

// Constructor dengan parameter nama
public Dataset(String nama) {
    this.nama = nama;
    totalDataset++;
}

// Constructor lengkap
public Dataset(String nama, int jumlahBaris, int jumlahKolom, int
jumlahMissing) {
    this.nama = nama;
    this.jumlahBaris = jumlahBaris;
    this.jumlahKolom = jumlahKolom;
    this.jumlahMissing = jumlahMissing;
    totalDataset++;
}

// Getter dan setter nama
public String getNama() {
    return nama;
}

public void setNama(String nama) {
    this.nama = nama;
}

// Getter dan setter jumlah baris
public int getJumlahBaris() {
    return jumlahBaris;
}

public void setJumlahBaris(int jumlahBaris) {
    if (jumlahBaris >= 0) {
```

```

        this.jumlahBaris = jumlahBaris;
    }
}

// Getter dan setter jumlah kolom
public int getJumlahKolom() {
    return jumlahKolom;
}

public void setJumlahKolom(int jumlahKolom) {
    if (jumlahKolom >= 0) {
        this.jumlahKolom = jumlahKolom;
    }
}

// Getter dan setter jumlah missing
public int getJumlahMissing() {
    return jumlahMissing;
}

public void setJumlahMissing(int jumlahMissing) {
    if (jumlahMissing >= 0) {
        this.jumlahMissing = jumlahMissing;
    }
}

// Menghitung missing
public double getPersentaseMissing() {
    int totalSel = jumlahBaris * jumlahKolom;

    if (totalSel == 0) {
        return 0;
    }

    return (double) jumlahMissing / totalSel * 100;
}

// Mengecek apakah dataset perlu dibersihkan
public boolean perluDibersihkan() {
    return getPersentaseMissing() > BATAS_MISSING;
}

// Mengambil total dataset tanpa membuat objek
public static int getTotalDataset() {
    return totalDataset;
}

```

```

    }
}

package project.unguided.laporan;

import project.unguided.model.Dataset;

public class LaporanDataset {

    public void cetak(Dataset dataset) {

        System.out.println("=== Laporan Dataset ===");
        System.out.println("Nama          : " + dataset.getNama());
        System.out.println("Jumlah Baris : " +
dataset.getJumlahBaris());
        System.out.println("Jumlah Kolom : " +
dataset.getJumlahKolom());

        System.out.printf(
            "Missing      : %d sel (%.2f%%)%n",
            dataset.getJumlahMissing(),
            dataset.getPersentaseMissing()
        );

        if (dataset.perluDibersihkan()) {
            System.out.println("Status          : Perlu dibersihkan");
        } else {
            System.out.println("Status          : Bersih");
        }

        System.out.println();
    }
}

```

```

package project.unguided.main;

import project.unguided.model.Dataset;
import project.unguided.laporan.LaporanDataset;

public class Main {

    public static void main(String[] args) {

        // constructor tanpa parameter
        Dataset dataset1 = new Dataset();
        dataset1.setNama("Titanic");
        dataset1.setJumlahBaris(891);
    }
}

```

```
dataset1.setJumlahKolom(12);
dataset1.setJumlahMissing(866);

//constructor dengan satu parameter
Dataset dataset2 = new Dataset("Wine Quality");

// constructor lengkap
Dataset dataset3 = new Dataset("Iris", 150, 5, 0);

// Menyimpan objek dalam array
Dataset[] daftarDataset = {
    dataset1,
    dataset2,
    dataset3
};

// Membuat objek laporan
LaporanDataset laporan = new LaporanDataset();

// Menampilkan laporan
for (Dataset dataset : daftarDataset) {
    laporan.cetak(dataset);
}

    System.out.println("Total dataset dibuat : " +
Dataset.getTotalDataset());
}
}
```

Output :

```
=== Laporan Dataset ===
Nama      : Titanic
Jumlah Baris : 891
Jumlah Kolom : 12
Missing    : 866 sel (8,10%)
Status     : Perlu dibersihkan

=== Laporan Dataset ===
Nama      : Wine Quality
Jumlah Baris : 0
Jumlah Kolom : 0
Missing    : 0 sel (0,00%)
Status     : Bersih

=== Laporan Dataset ===
Nama      : Iris
Jumlah Baris : 150
Jumlah Kolom : 5
Missing    : 0 sel (0,00%)
Status     : Bersih

Total dataset dibuat : 3
```

Penjelasan :

Package *model* digunakan untuk membuat class *Dataset* yang menyimpan data dataset serta menyediakan constructor, getter, dan setter untuk mengatur nilai atributnya. Package *laporan* berisi class *LaporanDataset* yang digunakan untuk mengambil data dari objek *Dataset*, menghitung persentase data missing, dan menentukan apakah dataset perlu dibersihkan. Sedangkan package *main* berisi class *Main* yang membuat tiga objek dataset dengan constructor yang berbeda, memasukkan objek tersebut ke dalam array, kemudian memanggil method *cetak()* untuk menampilkan hasil laporan setiap dataset.

D. Kesimpulan

Berdasarkan praktikum ini, dapat disimpulkan bahwa enkapsulasi digunakan untuk membatasi akses langsung terhadap atribut dengan memanfaatkan access modifier seperti *private* serta getter dan setter. Selain itu, juga mempelajari penggunaan constructor, termasuk constructor overloading untuk membuat objek dengan parameter yang berbeda, serta penggunaan package untuk mengelompokkan class agar program lebih terstruktur.

Dari yang telah dilakukan, penggunaan modifier, constructor, dan package dapat membantu dalam mengatur data dan hubungan antar-class pada program java. Penerapan konsep tersebut juga membuat kode menjadi lebih terorganisir, terumata ketika terdapat beberapa class yang memiliki fungsi berbeda tetapi saling berhubungan.

